Infineon/BIT Excellence Program 2026

Sustainable Supply Chain for Dried Mango from Africa to Germany

Sprint Note

Week of August 17–21, 2026 — Prepared by MADIEGA S Aida Justine

---

Summary

Backend implementation for the Energy Supply Systems module started this week, following the tech stack confirmed by the team (Spring Boot, PostgreSQL, Lombok). The module's data model was translated into working code aligned with the repository's existing conventions, and an improvement to the app mock-up was identified for the What-If Scenario Analysis tab.

This Week's Progress

- Tech stack confirmed and applied: Spring Boot, PostgreSQL, Lombok, React (JavaScript, JSX)
- Repository conventions reviewed (architecture, module ownership, git workflow) to align this module's code with the rest of the team
- Backend scaffolding started for modules/energysupply/: entities and repositories in place, following the layered structure (controller / service / repository / entity / dto) used across all nine modules
- Mock-up improvement identified: Added dynamic charts (Recharts) to the What-If tab so scenario changes update the visualization in real time
- Currently building Spring Boot proficiency step by step; open to a technical sync if useful, as was arranged for other modules

Next Steps

- Continue backend implementation: service and controller layers for the Energy Component entity
- Verify local setup via Docker Compose once enough backend code is in place to test

Sprint Note

Week of August 24–28, 2026 — Prepared by MADIEGA S Aida Justine

---

Summary

Finalization of the Energy Supply module backend: the entire scope requested by this week's feedback (*"Take the dummy data, save them to the database, connect to the dashboard"*) is now delivered. A complete package of Java files has been created, including all entities.

The Repository/Service/Controller layers, and an automated seeder that injects the Base scenario data into PostgreSQL at startup. The aggregated dashboard endpoint now returns values fetched from the database, and the foundation is ready to be merged into the main repository.

This Week's Progress

- Completed the remaining entities (Repository + Service + Controller) following the already validated model and the repository's conventions (layered architecture: controller / service / repository / entity / dto).
- Added a seeder (EnergySupplyDataSeeder) that automatically inserts at startup: 5 components, 6 daily consumption records, 1 monthly carbon metric, and 1 generator event directly addressing the need to inject the Excel data into the database.
- Developed controllers exposing full CRUD operations on the 6 tables, plus an aggregated endpoint dedicated to the dashboard:
  GET /api/energy/dashboard/summary → now returns real data (no more static mocks).
- Verified the consistency of Java files (no syntax or brace errors, everything is coherent).
- Identified a point of attention: two values (PV capacity in kWp and CO_2 avoided) were left as TODO in the seeder because the source Excel file was no longer accessible after the container reset.

Next Steps

- Prepare the package ready for deployment: merge the modules/energysupply/ folder, quick test via docker compose up -d --build, and validation of endpoint visibility in Swagger
- Confirm the two exact values from the Base scenario Excel file (search for // TODO: confirm from Excel in EnergySupplyDataSeeder.java) and update them before the demo.
- **Start** the frontend with react.

Sprint Note

Week of August 31–4 September, 2026 — Prepared by MADIEGA S Aida Justine

---

Summary

Development of the React dashboard for the energy supply tracking module. The interface now displays the key system indicators and connects to the backend API to retrieve real data, replacing static mock data with a functional backend integration.

This Week's Progress

- Developed the React dashboard displaying key indicators: solar capacity, consumption, generator status, and CO_2 avoided.
- Integrated the dashboard with the backend energy API (GET /api/energy/dashboard/summary) to fetch real-time data.
- Added a solar production chart covering the last 7 days.
- Added a component status table listing the individual state of each system component.

Next Steps

- Test the dashboard display with real data from the PostgreSQL database (once the two TODO values in the backend seeder are confirmed).
- Add tests to verify correct rendering of components (chart, table, indicators) across different data scenarios.

Sprint Note

Week of September 7–11, 2026 — Prepared by MADIEGA S Aida Justine

---

Summary

Following feedback to be more specific on the project task in progress, this week's focus was building EnergyApp.jsx, the main React frontend shell for the Energy Supply Systems module, and validating the full data pipeline end-to-end on its first tab. The Dashboard tab, connected to the backend energy API, was used as the reference implementation to confirm that data stored in PostgreSQL flows correctly through the Spring Boot backend into the React frontend before the same pattern is replicated across the remaining six tabs.

This Week's Progress

- Built EnergyApp.jsx: the module's main container, including sidebar navigation across all 7 tabs (Dashboard, Solar & Generation, Consumption Tracking, Generator Backup, Carbon Footprint, What-If Analysis, Alerts & Notifications), farm logo placement, alert badge, and tab-switching logic.

- Connected the Dashboard tab specifically to the backend through a dedicated API layer (energyApi.js), fetching live data from GET /api/energy/dashboard/summary, GET /api/energy/components, and GET /api/energy/solar/last-7-days.
- Identified and fixed a CORS misconfiguration blocking requests from the React frontend to the Spring Boot backend added @CrossOrigin on all 7 controllers.
- Validated the full stack end-to-end: confirmed that a change made directly in PostgreSQL reflects automatically on the Dashboard tab without redeployment, satisfying the requirement that displayed data be genuinely dynamic rather than static.
- Debugged and resolved two blocking issues encountered during testing: a backend compilation failure caused by a missing import, and a frontend styling issue caused by an incomplete Tailwind theme import.

Next Steps

- Resolve the two pending TODO values in EnergySupplyDataSeeder.java (PV capacity in kWp, CO_2 avoided), carried over from the previous sprint, once the original Excel business case figures are confirmed.
- Build the remaining six tabs inside EnergyApp.jsx , Solar & Generation, Consumption Tracking, Generator Backup, Carbon Footprint, What-If Analysis, Alerts & Notifications currently placeholders, following the now-validated Dashboard pattern.

Sprint Note

Week of September 14–18, 2026 — Prepared by MADIEGA S Aida Justine

Summary

Following feedback to explain all abbreviations, technical terms, and tools on the project homepage, this week's focus was building the first three of the six remaining frontend tabs and closing the two pending TODO values in the backend seeder with properly sourced figures, replacing last year's unverified data.

This Week's Progress

- Built three of the six remaining tabs in the Energy Supply module frontend: Solar & Generation, Consumption Tracking, and Generator Backup, each connected to the real backend API endpoints (not the original mock-up data) and aligned with the team's validated Tailwind theme tokens.

- Reviewed the module's real backend structure (controllers, entities, repositories, services) and the actual energyApi.js file to correct earlier assumptions about endpoint paths and available data fields.

- Identified that some fields from the original visual mock-up (fuel level percentage, critical loads table, trees-equivalent metric, emission factors by source) have no corresponding table in the real data model, and adjusted the affected tabs to only display data the backend can actually provide.

- Received a develop-branch update notice (frontend infrastructure setup merged in) requiring a manual sync of the module branch before further work.

- Resolved the two pending TODO values in EnergySupplyDataSeeder.java:

    - Sourced Burkina Faso's average peak sun hours (5.5 kWh/m²/day) from Nana et al. (2023), *Clean Air Journal*.

    - Sourced SONABEL's grid CO_2 emission factor (0.569 kg CO_2/kWh, based on the 2018 electricity mix) from the same study.

    - Calculated a PV capacity of approximately 8.2 kWp and CO_2 avoided of approximately 6,550 kg/year for the Base scenario, replacing the unsourced figures carried over from last year's file.

Next Steps

- Build the remaining three tabs: Carbon Footprint, What-If Analysis, and Alerts & Notifications.
- Merge the latest develop branch into the module branch to pick up the shared frontend infrastructure update.

Sprint Note

Week of September 21–24, 2026 — Prepared by MADIEGA S Aida Justine

Summary

This week's focus was completing a fourth frontend tab, resolving the last pending TODO value in the backend seeder and integrating the module's branch with the team's updated develop branch ahead of opening the module's first Pull Request.

This Week's Progress

- Built the Carbon Footprint tab, bringing the module's frontend to 4 of 6 tabs complete, connected to the real backend API and validated theme

- Rebased the module branch onto the latest develop, resolving two conflicts: App.jsx (removed in favor of the team's new shared router) and index.css where the module's theme collided with shared design tokens already used by other modules. Kept the shared tokens unchanged rather than overriding them unilaterally and flagged the resulting visual difference (teal instead of the originally validated green) for team discussion.

- Identified and resolved a third pending TODO in EnergySupplyDataSeeder.java: battery capacity, sized at 42.5 kWh from the Base scenario's daily load and a standard 80% depth-of-discharge assumption, with a 1-day autonomy design choice flagged for team confirmation.

- Corrected the CO_2 avoided figure from an annual to a monthly value (538.80 kg/month), since the underlying data model records this metric per month and corrected the associated emission factor to match the sourced value used elsewhere.

- Opened the module's first Pull Request into develop, covering the four completed tabs and all three resolved seeder TODOs, with sources documented in the PR description.

- Received feedback on how the backend will receive and use consumption data from other modules to improve load forecasts, and outlined a response: using internal Spring events so other modules (e.g. Machinery) can publish equipment energy usage without direct coupling between modules.

Next Steps

- Build the remaining two tabs: What-If Analysis and Alerts & Notifications.
- Wire the module into the team's shared router (routes.jsx is currently empty by design, to keep this PR small).
- Confirm the WhatIfResponseDto fields with the backend.
- Follow up on the index.css design token collision with the team.

Sprint Note

Week of September 28 – October 24, 2026 — Prepared by MADIEGA S Aida Justine

Summary

This week's focus was building the fifth frontend tab (What-If Analysis), fixing the CI failure on the module's first Pull Request, and restructuring that PR into smaller, single-topic PRs following the reviewer's request and the team's convention.

This Week's Progress

- Built the What-If Analysis tab, bringing the module's frontend to 5 of 6 tabs complete. While checking the component against the backend, found that the field names used in the frontend did not match the real WhatIfResponseDto (for example newPaybackYears instead of paybackYears) and that the backend returns no baseline values. Aligned the component with the real DTO and for this iteration, display only the projected outcome instead of a "Current vs Proposed" comparison, to avoid duplicating business logic in the frontend. Also added the € symbol to the CAPEX display.

- Diagnosed and fixed the CI failure on the first Pull Request: a Spring bean name clash between the Energy Supply DashboardController and the one in Visitor Management. Fixed by giving the bean an explicit name.

- Received review feedback that the Pull Request was too large to review (about 50 files, against the team's agreement of 20-30 files maximum, one topic at a time). Planned a split into four thematic PRs: (1) domain model (entities and repositories), (2) backend services, DTOs, controllers and seeder, (3) frontend theme and API client, (4) UI components. Each one is based on develop or on the previous PR, so reviewers only see the files of one topic.

- Rebased the module branch again on the latest develop. The diff first showed deletions in the plant's module, which turned out to be a teammate's harvest, planting and yield forecast code added after the previous rebase. Confirmed that the branch only touches energysupply files.

- Restored index.css to the develop version, so the module no longer modifies any shared file.

Next Steps

- Open the split PRs, link them in the old PR, then close it. Address review comments.
- Build the last tab: Alerts and Notifications.
- Wire the six tabs and the module into the team's shared router (routes.jsx is still empty).
- Add baseline values to the What-If backend (WhatIfService and DTO) to restore the Current vs Proposed comparison.

Sprint Note

Week of October 3 – October 9, 2026 — Prepared by MADIEGA S Aida Justine

Summary

This week's focus was acting on the reviewer's feedback on the Energy Supply PR and getting the module ready to merge into develop. Following the team's branch policy, the split into several PRs was replaced by a single consolidated PR from feature/energysupply/init. The planned work on the router, the What-If baselines and the Alerts tab did not start this week

This Week's Progress

- **Review feedback:** The reviewer closed the oversized PR and the first split PR because they targeted main instead of develop. I corrected the base branch and prepared a clearer PR description.

- **Branch policy:** While preparing the split PRs, I found the team's rule that PRs are only accepted from feature/<module>/init branches. This made the four-PR split impossible, so I consolidated all the work on feature/energysupply/init.

- **Branch cleanup:** The local branch had picked up plants-module files by accident. I realigned it with the remote branch, which only touches energysupply files.

- **Sync with develop:** I merged the latest develop and resolved two conflicts. App.jsx was deleted on develop, so I deleted it here. In index.css I kept the shared theme (teal primary, green secondary) so the module changes no shared styling. A merge was used instead of a rebase because the branch is protected against force-push. The backend compiles

- **CI and What-If fixes:** The Dashboard bean name fix and the What-If field alignment from last week are now part of the consolidated branch.

- **Test run:** A full local run on the merged branch shows 20 errors, all from Spring failing to load its context in other modules' tests (Cors, Visitor Management, Water Supply). develop alone passes with 582 tests, so I am tracing the cause. I suspect a name collision with the newly merged Water Supply module and I will fix it inside my module only

Next Steps

- Resolve the 20 test errors, then open the PR from feature/energysupply/init to develop with a description that explains the PR's size.
- Ask teammates to review the PR and run the module in their own environment, to confirm everything works together.
- Add baseline values to the What-If backend (WhatIfService and DTO) to restore the Current vs Proposed comparison.
- Wire the six tabs and the module into the shared router (routes.jsx is still empty).
- Verify the Alerts and Notifications tab against the real API.